Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЁТ
к лабораторной работе №7
на тему
«MONGODB ФРЕЙМВОРК АГРЕГАЦИИ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Реализовать агрегационные конвейеры для вычисления бизнес-метрик: ежемесячная выручка по категориям, анализ рыночной корзины (Market Basket Analysis) и RFM-анализ.

2 Реализовать механизм создания материализованных представлений с использованием стадии *$merge* для оптимизации аналитических запросов.

3 Провести сравнительный анализ производительности между выполнением агрегации в реальном времени и чтением из материализованного представления.

4 Реализовать сложный бизнес-процесс перемещения запасов между складами с сохранением истории операций.

5 Исследовать уровни изоляции и механизмы предотвращения состояний гонки (race conditions) при обновлении документов.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Фреймворк агрегации

Фреймворк агрегации в *MongoDB* основан на концепции конвейера (pipeline). Документы проходят через последовательность стадий, каждая из которых выполняет определенную трансформацию данных. Это позволяет выполнять сложную аналитическую обработку на стороне сервера БД.

Основные стадии, использованные в работе:

– *$lookup* – выполнение левого внешнего соединения (left outer join) с другой коллекцией;

– *$unwind* – деконструкция массива в документах для обработки каждого элемента отдельно;

– *$group* – группировка документов по заданному ключу и вычисление агрегатных значений;

– *$addFields* – добавление новых полей в документы;

– *$project* – изменение структуры документов, выбор и переименование полей;

– *$merge* – запись результатов конвейера в коллекцию (используется для материализованных представлений).

2.2 Материализованные представления и стадия $merge

Материализованные представления позволяют кэшировать результаты тяжелых агрегационных запросов. Стадия *$merge* позволяет инкрементально обновлять целевую коллекцию, объединяя или заменяя документы на основе уникального ключа. Это значительно ускоряет чтение аналитических данных за счет переноса вычислительной нагрузки на этап записи или регламентного обновления.

2.3 Атомарность и изоляция

В *MongoDB* операции записи атомарны на уровне одного документа. Для предотвращения проблем конкурентного доступа (например, *Lost Update*) используются атомарные операторы обновления (как *$inc*) в сочетании с условиями в запросе (фильтрами). Это позволяет реализовать оптимистическую блокировку без явного использования транзакций в простых сценариях.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Агрегационные конвейеры для бизнес-метрик

3.1.1 Ежемесячная выручка по категориям

Для расчета выручки выполняется соединение коллекций заказов, позиций, товаров и категорий. Используется оператор *$substr* для извлечения месяца из даты.

Листинг 3.1 – GetMonthlyRevenueByCategory (Агрегация выручки)

```
db.orders.aggregate([
  { $lookup: { from: "orderItems", localField: "_id", foreignField:
"orderId", as: "items" }},
  { $unwind: "$items" },
  { $lookup: { from: "products", localField: "items.productId",
foreignField: "_id", as: "product" }},
  { $unwind: "$product" },
  { $lookup: { from: "categories", localField: "product.categoryId",
foreignField: "_id", as: "category" }},
  { $unwind: "$category" },
  { $project: {
      month: { $substr: ["$createdAt", 0, 7] },
      categoryName: "$category.name",
      revenue: { $multiply: ["$items.quantity", "$items.price"] }
  }},
  { $group: {
      _id: { month: "$month", category: "$categoryName" },
      totalRevenue: { $sum: "$revenue" }
  }},
  { $sort: { "_id.month": 1, totalRevenue: -1 }}
])
```

3.1.2 Анализ рыночной корзины (Market Basket Analysis)

Задача состоит в поиске пар товаров, которые часто покупают вместе. Конвейер ищет заказы с более чем одним товаром и формирует уникальные пары.

Листинг 3.2 – GetMarketBasketAnalysis (Поиск пар товаров)

```
db.orders.aggregate([
  { $lookup: { from: "orderItems", localField: "_id", foreignField:
"orderId", as: "items" }},
  { $match: { "items.1": { $exists: true } }}, // Минимум 2 товара
  { $project: { productIds: "$items.productId" }},
  { $unwind: "$productIds" },
  { $lookup: { from: "orderItems", localField: "_id", foreignField:
"orderId", as: "otherItems" }},
  { $unwind: "$otherItems" },
  { $project: { p1: "$productIds", p2: "$otherItems.productId" }},
  { $match: { $expr: { $lt: ["$p1", "$p2"] } }}, // Исключаем
```

```
дубликаты (A,B == B,A) и самопересечения
  { $group: {
      _id: { p1: "$p1", p2: "$p2" },
      count: { $sum: 1 }
  }},
  { $sort: { count: -1 }},
  { $limit: 10 }
])
```

3.1.3 RFM-анализ

Анализ лояльности клиентов по трем показателям: *Recency* (давность), *Frequency* (частота), *Monetary* (деньги).

Листинг 3.3 – GetRFMAnalysis (Сегментация клиентов)

```
db.orders.aggregate([
  { $lookup: { from: "orderItems", localField: "_id", foreignField:
"orderId", as: "items" }},
  { $addFields: {
      orderTotal: { $sum: {
          $map: {
              input: "$items", as: "i",
              in: { $multiply: ["$$i.quantity", "$$i.price"] }
          }
      }}
  }},
  { $group: {
      _id: "$userId",
      lastOrderDate: { $max: "$createdAt" },
      frequency: { $sum: 1 },
      monetary: { $sum: "$orderTotal" }
  }},
  { $sort: { monetary: -1 } }
])
```

3.2 Анализ производительности и материализованные представления

Для оптимизации запроса выручки создано материализованное представление с использованием стадии *$merge*.

Листинг 3.4 – CreateMonthlyRevenueMaterializedView (Использование $merge)

```
// Конец конвейера агрегации выручки дополняется стадией $merge
{ $merge: {
    into: "mv_monthly_revenue",
    whenMatched: "replace",
    whenNotMatched: "insert"
}}
```

Сравнение времени выполнения показало значительное преимущество материализованного представления. При чтении готовых агрегатов исключается необходимость выполнения множественных соединений (*$lookup*) и группировок над всем объемом данных заказов. Разница во времени выполнения отображена на рисунке 3.1

```
=== Performance Comparison: Live vs Materialized ===
Live Aggregation Time: 4.108114ms
Materialized View Created ($merge)
Materialized View Read Time: 473.668µs
Speedup: 8.67x
```

Рисунок 3.1 – Результаты сравнения производительности

3.3 Сложный бизнес-процесс: перемещение запасов

Реализован процесс перевода товара между складами. Операция включает уменьшение остатка на одном складе, увеличение на другом и запись в историю перемещений.

Листинг 3.5 – TransferStock (Перемещение между складами)

```
func (s *Service) TransferStock(ctx context.Context, productID int,
from, to string, qty int) error {
    invColl := s.mdb.Collection("inventory")
    // Уменьшение остатка (from)
    invColl.UpdateOne(ctx,
        bson.D{{"productId", productID}, {"warehouse", from}},
        bson.D{{"$inc", bson.D{{"quantity", -qty}}}},
        options.UpdateOne().SetUpsert(true),
    )
    // Увеличение остатка (to)
    invColl.UpdateOne(ctx,
        bson.D{{"productId", productID}, {"warehouse", to}},
        bson.D{{"$inc", bson.D{{"quantity", qty}}}},
        options.UpdateOne().SetUpsert(true),
    )
    // Запись в историю
    s.mdb.Collection("inventoryHistory").InsertOne(ctx, bson.D{
        {"productId", productID}, {"from", from}, {"to", to},
{"quantity", qty},
        {"timestamp", time.Now().Format(time.RFC3339)},
    })
    return nil
}
```

3.4 Анализ уровней изоляции и предотвращение Race Condition

Для предотвращения продажи товара, которого нет в наличии, при одновременных запросах используется атомарное обновление с условием. Запрос проверяет, что `quantity > 0` перед тем, как уменьшить его на 1.

Листинг 3.6 – BuyLastItem (Атомарная проверка остатка)

```
func (s *Service) BuyLastItem(ctx context.Context, productID int, user
string) error {
    invColl := s.mdb.Collection("inventory")
    res, err := invColl.UpdateOne(ctx,
        bson.D{
            {"productId", productID},
            {"quantity", bson.D{{"$gt", 0}}}, // Проверка условия
        },
        bson.D{{"$inc", bson.D{{"quantity", -1}}}}, // Атомарный
декремент
    )
    if res.ModifiedCount == 0 {
        return fmt.Errorf("item out of stock")
    }
    return nil
}
```

При симуляции одновременной покупки последнего товара двумя пользователями (Alice и Bob), один из них успешно совершает покупку, а второй получает ошибку «item out of stock», так как условие `quantity > 0` перестает выполняться сразу после первого обновления.

ВЫВОД

В ходе выполнения лабораторной работы были изучены продвинутые возможности *MongoDB Aggregation Framework*. Были реализованы сложные аналитические запросы, такие как RFM и Market Basket Analysis, с использованием многостадийных конвейеров, что позволило эффективно обрабатывать большие объемы данных на стороне сервера. Также была освоена техника оптимизации запросов через материализованные представления со стадией *$merge*, которая обеспечила ускорение получения данных более чем в 7 раз по сравнению с выполнением агрегации в реальном времени. Кроме того, были изучены механизмы обеспечения целостности данных при конкурентном доступе с использованием атомарных операций обновления и фильтрации по условию, что позволило успешно решить проблему состояний гонки без использования тяжеловесных транзакций. Разработанные решения подтверждают эффективность NoSQL-подхода для построения аналитических систем и управления сложными бизнес-процессами.